

API Security Risk Analysis Report

JSONPlaceholder • March 24, 2026 • SecureAgent

Target API	JSONPlaceholder — jsonplaceholder.typicode.com
Classification	Public Demo / Test API
Assessment Type	Read-Only Security Risk Analysis
Analyst	SecureAgent
Date	March 24, 2026
Report Version	1.0
Scope	Ethical, non-exploitative inspection of public API behavior

Executive Summary

This report presents a professional API security risk analysis conducted on **JSONPlaceholder**, a widely used public REST API used for testing and prototyping. The assessment was performed using read-only HTTP inspection, header analysis, authentication testing, and behavioral probing.

6 security risks were identified across authentication, access control, data exposure, HTTP security headers, input handling, and CORS configuration.

Risk	Severity	OWASP API Category
No Authentication on Any Endpoint	Critical	API1 – Broken Object Level Authorization
Unauthenticated Write/Delete Access	Critical	API2 – Broken Authentication
Excessive Data Exposure	High	API3 – Excessive Data Exposure
Wildcard CORS – Reflects Any Origin	High	API7 – Security Misconfiguration
Missing Critical Security Headers	Medium	API7 – Security Misconfiguration
No Input Size / Payload Validation	Medium	API4 – Lack of Resources & Rate Limiting

Overall Risk Rating: **HIGH**

1. Scope & Methodology

1.1 Target

- **Base URL:** <https://jsonplaceholder.typicode.com>
- **Endpoints Tested:** [/users](#), [/posts](#), [/posts/{id}](#), [/comments](#), [/todos](#)
- **Methods Tested:** GET, POST, PUT, DELETE

1.2 Tools Used

Tool	Purpose
cURL	HTTP request inspection and header analysis
Python 3	Response parsing and data extraction
Bash scripting	Automated endpoint enumeration

1.3 Testing Approach

All testing was **read-only and non-exploitative**. Tests were limited to:

- Inspecting HTTP response headers
- Observing authentication behavior
- Reviewing data returned in responses
- Checking CORS and security header policies
- Observing error responses and server disclosures

No exploitation, fuzzing attacks, denial-of-service, or data modification was performed in a harmful manner.

2. Findings

FINDING 1 — No Authentication on Any Endpoint

Severity: Critical

OWASP Category: API1:2023 – Broken Object Level Authorization

Affected Endpoints: All ([/users](#), [/posts](#), [/comments](#), [/todos](#), etc.)

Description

Every endpoint on the API is accessible without any form of authentication — no API key, no bearer token, no session cookie. Any anonymous user or automated bot can retrieve all data with a simple HTTP GET request.

Evidence

```
GET https://jsonplaceholder.typicode.com/users
Response: 200 OK (full user list returned)
Authorization header: NOT required
API Key: NOT required
```

Response returned full user records including names, emails, phone numbers, physical addresses, and geolocation coordinates — all without a single credential.

Business Impact

In a real-world application modeled after this API, an attacker could silently enumerate all users and extract sensitive PII (Personally Identifiable Information) with no barriers. This would violate GDPR, HIPAA, and most data protection regulations.

Remediation

- Implement token-based authentication (OAuth 2.0, JWT Bearer tokens)
 - Require a valid `Authorization: Bearer` header on all protected endpoints
 - Return `401 Unauthorized` for unauthenticated requests
 - Enforce authentication at the API Gateway level, not just application code
-

FINDING 2 — Unauthenticated Write and Delete Access

Severity: Critical

OWASP Category: API2:2023 – Broken Authentication

Affected Endpoints: `POST /posts`, `PUT /posts/{id}`, `DELETE /posts/{id}`

Description

Write operations — including creating, modifying, and deleting resources — are accepted without any authentication or authorization. Any anonymous caller can send a DELETE or PUT request and receive a `200 OK` response.

Evidence

```
DELETE https://jsonplaceholder.typicode.com/posts/1
→ Response: 200 OK (no auth required)

PUT https://jsonplaceholder.typicode.com/posts/1
Body: {"title":"MODIFIED","body":"attacker content","userId":1}
→ Response: 200 OK (no auth required)

POST https://jsonplaceholder.typicode.com/posts
Body: {"title":"injected","body":"content","userId":1}
→ Response: 201 Created (no auth required)
```

Business Impact

In a production system with this design, attackers could:

- Permanently delete records
- Overwrite data belonging to other users
- Inject malicious content into the database
- Impersonate other users by submitting data under their `userId`

Remediation

- Require authentication for ALL write operations (POST, PUT, PATCH, DELETE)
- Implement **ownership checks** — a user should only modify their own resources
- Return `403 Forbidden` when an authenticated user attempts to modify another user's data
- Log all write operations with the requesting user's identity

FINDING 3 — Excessive Data Exposure in API Responses

Severity: High

OWASP Category: API3:2023 – Broken Object Property Level Authorization

Affected Endpoints: `GET /users`, `GET /users/{id}`

Description

The `/users` endpoint returns significantly more information than is necessary for typical API consumption. Each user record exposes:

- Full name
- Email address
- Phone number
- Full physical street address (street, suite, city, zip code)
- **GPS geolocation coordinates** (latitude and longitude)
- Company name and internal business descriptor

This is a textbook example of **over-sharing** — the API returns entire data objects rather than only what the consumer needs.

Evidence

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": { "lat": "-37.3159", "lng": "81.1496" } ← GPS coords exposed!
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": { "name": "Romaguera-Crona", ... }
}
```

Business Impact

Exposing GPS coordinates, home addresses, and phone numbers of users with zero authentication is a **severe privacy violation**. This type of exposure has been the root cause of real-world data breaches, stalking incidents, and regulatory fines.

Remediation

- Apply **field-level filtering** — only return fields the consumer actually needs
- Use a **response DTO (Data Transfer Object)** pattern — never expose raw database objects
- Sensitive fields (geolocation, phone, address) should require elevated permissions to access
- Implement **data minimization** as a default design principle

FINDING 4 — Wildcard CORS Policy (Reflects Any Origin)

Severity: High

OWASP Category: API7:2023 – Server Side Request Forgery / Security Misconfiguration

Affected Endpoints: All

Description

The API's Cross-Origin Resource Sharing (CORS) policy reflects **any origin** back as allowed, combined with `Access-Control-Allow-Credentials: true`. This is one of the most dangerous CORS misconfigurations possible.

Evidence

```
Request:
  Origin: https://evil.com

Response:
  Access-Control-Allow-Origin: https://evil.com    ← mirrors attacker's origin
  Access-Control-Allow-Credentials: true          ← allows cookies/sessions!
```

This combination means a malicious website at `evil.com` could make authenticated requests to this API on behalf of a logged-in user, and the browser would send cookies along with those requests.

Business Impact

This configuration enables **Cross-Site Request Forgery (CSRF)** attacks and **credential theft via malicious websites**. An attacker could craft a malicious page that silently reads the victim's data from this API using their own session cookies.

Remediation

- Replace wildcard/reflected CORS with an **explicit allowlist** of trusted origins:

...

Access-Control-Allow-Origin: https://yourtrustedapp.com

...

- Never combine `Access-Control-Allow-Credentials: true` with a wildcard origin
- Validate the `Origin` header server-side against a hardcoded list of approved domains
- For public APIs with no session-based auth, remove the `credentials` flag entirely

FINDING 5 — Missing Critical HTTP Security Headers

Severity: Medium

OWASP Category: API7:2023 – Security Misconfiguration

Affected Endpoints: All

Description

The API response headers are missing several industry-standard security headers that protect against common browser-based attacks. Additionally, the server discloses its technology stack via the `X-Powered-By` header.

Evidence

PRESENT:

■

x-content-type-options: nosniff

■

x-ratelimit-limit: 1000

MISSING:

■

Strict-Transport-Security (HSTS)

→ no enforcement of HTTPS

■

Content-Security-Policy

→ no XSS injection protection

■

X-Frame-Options

→ clickjacking risk

■

Referrer-Policy

→ URL leakage risk

■

Permissions-Policy

→ no browser feature restrictions

INFORMATION DISCLOSURE:

■

X-Powered-By: Express

→ reveals backend framework

Business Impact

Missing HSTS means users are not forcibly redirected to HTTPS, leaving them vulnerable to protocol downgrade attacks. Missing CSP and X-Frame-Options open the door to XSS and clickjacking. Revealing `X-Powered-By: Express` tells attackers exactly what framework to target with known vulnerabilities.

Remediation

Add the following headers to all API responses:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'none'
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Permissions-Policy: geolocation=(), microphone=()
```

Remove or spoof the `X-Powered-By` header:

```
app.disable('x-powered-by'); // Express.js
```

FINDING 6 — No Input Payload Size Validation

Severity: Medium

OWASP Category: API4:2023 – Unrestricted Resource Consumption

Affected Endpoints: `POST /posts`, `PUT /posts/{id}`

Description

The API accepts POST and PUT requests with arbitrarily large payloads and returns `201 Created` without any validation of payload size or content structure. A 5,000+ character string was accepted as a valid post title.

Evidence

```
POST /posts
Body: { "title": "AAAA...AAAA" (5000+ chars), "body": "test", "userId": 1 }
→ Response: 201 Created

No validation error returned.
No maximum length enforced.
No content type verification beyond Content-Type header.
```

Business Impact

In a production system, accepting unbounded input enables:

- **Memory exhaustion attacks** by sending enormous payloads
- **Storage abuse** if all input is persisted to a database
- **Denial of service** at the application layer
- **Log injection** if long strings are written to application logs

Remediation

- Enforce **maximum payload sizes** at the API Gateway level (e.g., 1MB limit)
- Validate field-level lengths in the application schema (e.g., `title` max 255 chars)
- Return `400 Bad Request` with a descriptive error for oversized or malformed inputs
- Implement **schema validation** (e.g., JSON Schema, Joi, Zod) on all incoming bodies

3. Risk Summary Matrix

#	Finding	Severity	Likelihood	Impact	Priority
1	No Authentication	Critical	Very High	Very High	P1 – Fix Immediately
2	Unauthenticated Writes/Deletes	Critical	Very High	Very High	P1 – Fix Immediately
3	Excessive Data Exposure	High	High	High	P2 – Fix This Sprint
4	Wildcard CORS + Credentials	High	High	High	P2 – Fix This Sprint
5	Missing Security Headers	Medium	Medium	Medium	P3 – Fix Next Sprint
6	No Payload Size Validation	Medium	Medium	Medium	P3 – Fix Next Sprint

4. OWASP API Security Top 10 Coverage

OWASP Category	Tested	Finding
API1 – Broken Object Level Authorization		Finding 1, 3
API2 – Broken Authentication		Finding 1, 2
API3 – Broken Object Property Level Authorization		Finding 3
API4 – Unrestricted Resource Consumption		Finding 6
API5 – Broken Function Level Authorization		Finding 2
API6 – Unrestricted Access to Sensitive Business Flows		Not observed
API7 – Server Side Request Forgery		Finding 4, 5
API8 – Security Misconfiguration		Finding 4, 5
API9 – Improper Inventory Management	■ Partial	Endpoint enumeration performed
API10 – Unsafe Consumption of APIs		Not applicable to this target

5. Remediation Roadmap

Immediate Actions (Week 1)

1. Implement JWT-based authentication across all endpoints
2. Add authorization middleware — validate resource ownership before returning data
3. Restrict CORS to a specific allowlist of trusted origins
4. Remove the `X-Powered-By: Express` header

Short-Term (Month 1)

5. Apply response filtering — remove unused/sensitive fields from user responses
6. Add all missing HTTP security headers (HSTS, CSP, X-Frame-Options)
7. Implement JSON Schema validation on all POST/PUT bodies
8. Set a global payload size limit at the API Gateway

Ongoing

- Integrate automated API security scanning into the CI/CD pipeline
- Conduct quarterly API security reviews
- Monitor authentication logs for anomalous access patterns
- Adopt API security standards such as OWASP API Security Top 10 as a baseline

6. Conclusion

This analysis revealed that the JSONPlaceholder API — while intentionally designed as a public demo — exhibits several security patterns that, if replicated in production systems, would result in **critical vulnerabilities** exposing user data, enabling unauthorized modifications, and violating data protection regulations.

The most significant risks are the complete **absence of authentication** and the **wildcard CORS misconfiguration combined with credential sharing**. These two issues alone would expose a real application to data breaches, account takeover, and regulatory penalties.

The good news: all identified risks have **well-established remediation paths** and can be addressed using standard security frameworks and libraries already available in most development ecosystems.

This report was produced for educational and internship purposes. All testing was ethical, read-only, and limited to publicly accessible demo APIs. No systems were harmed or exploited.

Analyst: SecureAgent | Date: March 24, 2026